SOCAL WILDFIRES GRID CO-SIMULATION

A validated extreme-event testbed for infrastructure resilience

Eric M. S. P. Veith (OFFIS) Vivian Sultan (Cal State LA)

Onboarding for new project members

MOTIVATION

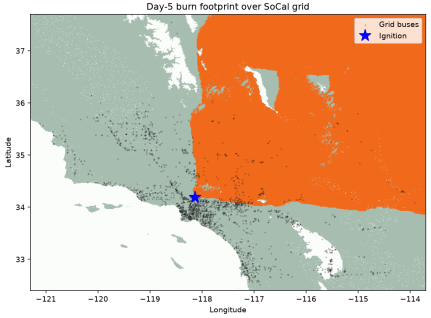
Why This Project Exists

January 2025, Los Angeles basin

- The **Eaton** and **Palisades** fires burned through the LA basin under Santa-Ana forcing.
- Wind-aligned fronts advanced at **60–80 m/min**, with spotting beyond 1 km.
- Distribution circuits, communication links and control systems were overwhelmed *simultaneously*.
- Consequence was not just burned area — it was **loss of electric service** to hundreds of thousands of customers.

The chain this project models

fire spread → asset damage → grid state → resilience metrics



Simulated fire perimeter, day 5

(`analysis/fire_perimeter_day5.png`)

The Research Gap

Fire models and grid models rarely talk to each other

Fire-centric	Grid-centric	This testbed
Detailed spread physics (Rothermel, Cell2Fire, FARSITE).	Detailed power-flow and contingency analysis.	Validated CA spread *and* full power-flow re-solve, every step, on **one shared geospatial frame**.
Infrastructure is *absent* or a static value layer.	Fire enters as an *exogenous* outage list or PSPS scenario.	A burned cell and its electrical asset are always co-registered.

- Mao et al. (2026) explicitly call for *feedback-coupled fire/grid simulations*.
- Without coupling you cannot ask: *did that suppression tactic protect the right cells?*

Source: Veith & Sultan, "When Wildfire Breaks the Grid", Sec. 1–2.

The problem

- DRL controllers are trained on data they have seen.
- Catastrophic events are, by definition, **rare** — they sit outside the training distribution.
- So agents fail *exactly* when resilience matters most. This is the **Curse of Rarity**.

The GUARDIAN answer

- Synthesise physically plausible worst-case worlds from curated GIS data.
- Train foundation policies against them via **Adversarial Resilience Learning (ARL)**.

Project facts

NATO Science for Peace and Security Multi-Year Project.

Pillars: energy security, environmental hazards, advanced AI.

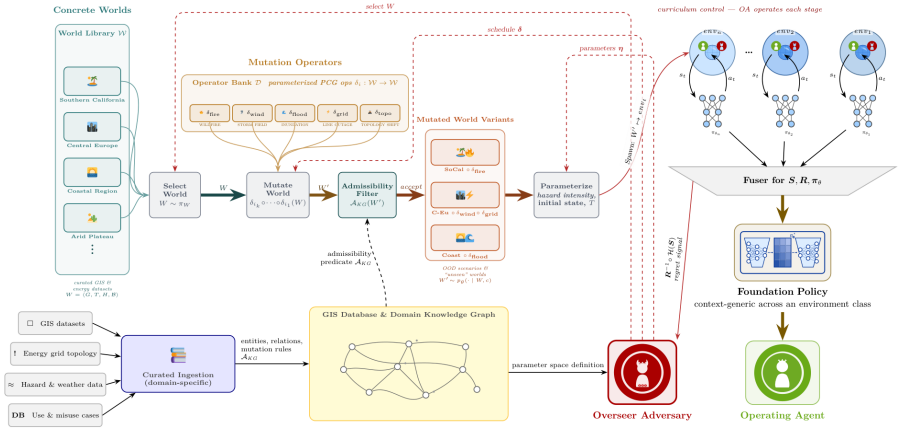
Dual PI: **Veith** (OFFIS, DE) & **Sultan** (Cal State LA, US).

Validation cases: SoCal wildfire cascades; Central European severe-weather grid failures.

Motivating precedent: the Feb. 2022 KA-SAT cyberattack disabled 5,800 wind turbines (≈11 GW) across Central Europe.

GUARDIAN System Architecture

Where this repository sits in the bigger picture



This repository implements the *SoCal* $\circ\, \delta_{fire}$ branch: one concrete world, one mutation operator, one operating agent.

Figure 2 from Sultan, Logemann & Veith, "GUARDIAN".

What You Will Take Away

Learning objectives

1. **Purpose** — what question this testbed is built to answer, and what it deliberately does *not* model.
2. **Concepts** — the constrained-mutation formalism CMA $= (S, \tau, D, \Theta)$ and where learning enters.
3. **Architecture** — how the geospatial, fire, grid and agent layers fit together in palaestrAI.
4. **Navigation** — which directory and which module to open for a given question.
5. **Operation** — the exact commands to reproduce every published figure.
6. **Research** — open ARL questions that are available as thesis topics.

Prerequisite

Basic palaestrAI familiarity. Slide 8 is a short refresher; a one-page reminder of experiment runs, environments and muscles.

PURPOSE AND CONCEPTUAL MODEL

What the Repository Delivers

Four deliverables

1 — NOAA-enriched MIDAS scenario

SoCal co-simulation scenario with weather switched from DWD Bremen to **NOAA HRRR** reanalysis, inside the scenario.

2 — palaestrAI environment

SoCal MIDAS wrapped as a first-class `palaestrai.environment.Environment`, plus an experiment run file.

3 — Wildfire agent (GUARDIAN CMA)

Cellular automaton driven by the Overseer-Adversary parameter vector Θ, over the full SoCal GIS footprint; optional PostGIS layer.

4 — Five-day co-simulation

120 h Santa-Ana wildfire/grid run with KPIs, plots and a generated report.

Source: repository `README.md`.

palaestrAI Refresher

The five nouns you need for this deck

Experiment run — A declarative YAML document: environments, agents, phases, termination conditions. Fully reproducible — the document *is* the experiment.

Environment — Owns state, publishes **sensors**, accepts **actuators**, is stepped by a simulation controller.

Agent — A **Brain** (learns, lives in the Learner process) plus one or more **Muscles** (act, live in RolloutWorkers).

Objective — Maps sensor readings to scalar reward. Separate from the environment, so several agents can score the same world differently.

Store — Every sensor, actuator and reward is written to a database, so analysis scripts read results instead of re-running simulations.

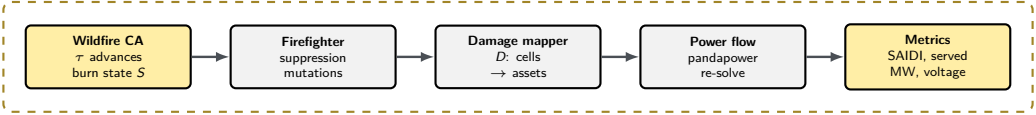
Two conventions used throughout

`TakingTurnsSimulationController` — agents act in a fixed order, not simultaneously.

`DummyBrain` — an agent that *acts but never learns*.

End-to-End Simulation Loop

One timestep



all four stages share one geospatial reference frame

- Agents act in **fixed turn order**: `wildfire` $\rightarrow$ `firefighter` $\rightarrow$ `damage_mapper`.
- Conflicting cell edits are resolved by *priority*, not by turn order — so the result is deterministic.

Mental Model: CMA $= (S, \tau, D, \Theta)$

The GUARDIAN constrained-mutation automaton

S — state

`int8` cell grid, co-registered with the raster. `UNBURNED` 0, `BURNING` 1, `BURNED_OUT` 2, `SUPPRESSED` 3, `CONTAINED` 5.

τ — transition

The CA sub-step (`dt_cma_min`, default 5 min). Each burning cell attempts to ignite its 8 Moore neighbours

D — damage map

Fails buses whose cell is burning or burned, and overhead lines inside a radiant-heat clearance buffer.

Θ — the adversary's handle

Ignition point(s), wind vector, dead-fuel moisture, and the global ROS multiplier κ ($1 \leq \kappa \leq 8$).

Θ **is the entire attack surface** — everything else is physics.

Source: `docs/CMA_AGENT.md`, `wildfire_cma/cma.py`.

What Is — and Is Not — Learning

The single most common misreading of this repo

Component	Brain	Learns?
Wildfire (GUARDIAN CMA)	`DummyBrain`	**No** — constrained mutation operator driven by Θ
Damage mapper	`DummyBrain`	**No** — deterministic consequence model
Firefighter v0.3–v0.5	scripted doctrine	**No** — expert incident-command heuristics
Firefighter v0.7	`FirefighterSacBrain`	**Yes** — SAC with CQL, bootstrapped offline

Why the fire does not learn

This is a deliberate, paper-faithful choice. The published work positions the CA as *the calibrated, validated substrate* on which a learned scenario-generation loop can *later* be built.

Why that matters to you

Turning Θ from a calibrated constant into a learned adversarial policy is the open door into ARL — and the subject of Sec. 6 of this talk.

TECHNICAL ARCHITECTURE

Repository Map

Where to look for what

Directory	Contents
`wildfire_cma/`	The CA itself: `cma.py` (S, τ, Θ), `gis.py`, `damage.py`, `wind_field.py`, `postgis.py`. Pure `numpy`, no palaestrAI import.
`palaestrai_socal/`	Environments (`gis_world_env.py`, `midas_grid_env.py`), all agents under `agents/`, and every `experiment*.yml`.
`socal_grid/`	Geo-referenced pandapower model plus `dispatch_and_run.py` (the convergence recipe) and `build_network.py`.
`midas_socal/`	MIDAS scenario `socal_midas.yml`, NOAA weather provider, `prepare_midas.py`, `run_sim.sh`.
`analysis/`	Drivers, report generators, timelapse builders and the committed result figures.
`data/`	GIS layers, DEM fetcher, CAL FIRE perimeters, offline teacher datasets, CAISO load.
`docs/, tests/`	Design docs; ≈30 test modules (**235 passing** as of v0.7).

Start reading at `wildfire_cma/cma.py` — it has no framework dependencies.

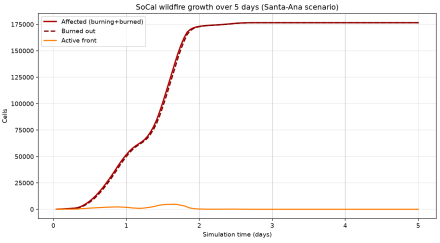
Geospatial Layer

The shared reference frame

- Footprint `SOCAL_BOUNDS` $= (-121.3,\ 32.4,\ -113.7,\ 37.7)$, EPSG:4326. Row 0 is the *north* edge.
- Three raster sources, in order of fidelity:
  - `from_rasters()` — real LANDFIRE FBFM40 fuel + 3DEP elevation (needs `rasterio`).
  - `socal_from_srtm()` — SRTM GL3 (≈90 m) via OpenTopography, 4×3 tile mosaic, resampled to a 600×760 model grid.
  - `synthetic_socal()` — deterministic fallback used by CI.
- Degrades *gracefully*: a missing DEM cache silently falls back to synthetic terrain.

Scale trap

On the statewide grid one cell ≈ 947 m. Retardant drops are sub-cell there — which is why fine grids (≈50 m) are used for suppression studies.



Fire growth on the statewide raster

(`analysis/fire_growth.png`)

Wildfire Spread Kernel

Rothermel-derived, wind- and slope-modulated

Rate of spread

$$R(i,j \to i',j') \;=\; \kappa \cdot R^{0}(i,j) \cdot \varphi_w(u,\theta) \cdot \varphi_s(z,\theta)$$

Ignition probability per sub-step

$$p \;=\; 1 - \exp(-\,R \cdot \Delta t_{\mathrm{CMA}}/\delta)$$

- R^0 — base ROS by fuel class, damped by dead-fuel moisture through the Rothermel coefficient η_m; zero above `FUEL_MX_EXTINCTION`.
- $\varphi_w = \exp(c\,u\cos\alpha)$, $c = 0.25$, tuned so $\approx$20 m/s Santa-Ana wind reproduces the observed **60–80 m/min** fronts.
- $\varphi_s = 1 + 5.275\tan^2(\text{slope})$· upslope alignment.
- δ = cell size in m ($\times\sqrt{2}$ diagonally). A neighbour ignites iff `rng.random() < p` — the only stochastic element, and it is seeded.

#	Fuel class	R^0 m/min
0	non-burnable	0.0
1	grass (GR)	4.0
2	grass-shrub (GS)	2.5
3	shrub / chaparral (SH)	**3.2**
4	timber-understory (TU)	1.2
5	timber-litter (TL)	0.8
6	slash-blowdown (SB)	1.5

Class 3 dominates SoCal. `gis.fbfm40_to_class()` maps the 40 Scott-Burgan models onto these seven.

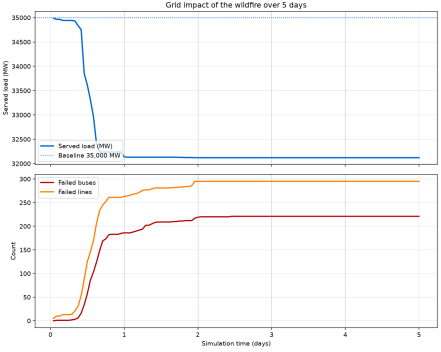
Power Grid Model

The electrical side

- **2,294 buses / 2,595 lines**, geo-referenced pandapower model (`socal_grid/socal_grid.json`, EPSG:4326, ≈5.9 MB).
- Built from open data: OSM substations, transmission lines, power plants, city populations, CAISO load profiles.
- An *illustrative* distribution topology — deliberately **not** a utility feeder model.
- Customers derived via `CUSTOMERS_PER_MW = 200.0`.

Convergence gotcha — read this first

Raw `pp.runpp()` on `socal_grid.json` **diverges**. Always go through `socal_grid/dispatch_and_run.py` — "config D": strengthen the network, co-locate dispatch, then solve with **Iwamoto Newton-Raphson**.



Grid impact over the five-day run

(`analysis/grid_impact.png`)

Damage Mapping

From burned cells to de-energised assets

The mechanism

1. Every bus and every line endpoint is projected onto the raster once, at setup.
2. A bus whose cell is `BURNING` or `BURNED_OUT` is de-energised.
3. An overhead line is removed if its corridor intersects a burning cell within the *radiant-heat clearance buffer*.
4. Power flow is re-solved; SAIDI and served MW are recorded.

Implementation note

mosaik exposes no `in_service` actuator for buses or lines. So under MIDAS the damage is applied as a **load-shed trip**: write `...load-<bus>-<idx>.p_mw` $\rightarrow 0$.

Mutation arbitration

Several agents may edit the same cell in one step. A fixed priority makes the outcome independent of turn order:

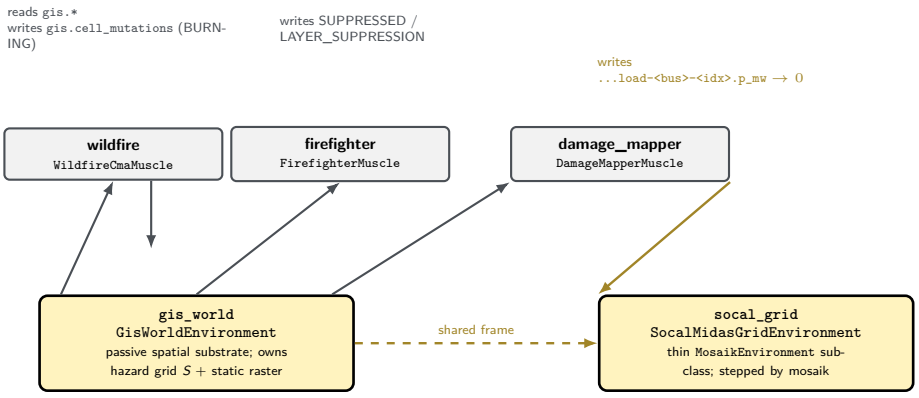
`BURNED_OUT` (terminal) $>$ `SUPPRESSED` $>$ `FLOODED` $>$ `BURNING` $>$ `UNBURNED`

See `spaces.arbitrate_mutations`. Suppression is not permanent: the environment reverts a cell after `SUPPRESS_PERSIST_STEPS` $= 12$.

Source: `docs/CMA_AGENT.md`, `docs/AGENTS.md`.

Multi-Agent Architecture

The v0.2 two-environment design



- `gis_world` is **hazard-agnostic**: it steps no dynamics, it only applies `gis.cell_mutations` and re-publishes telemetry. Wildfire today, flood or storm tomorrow.
- Ignition is an *agent parameter* (`ignition_points`, `ignition_step`), **not** an environment actuator.

Firefighting Model Evolution

Five releases, increasing realism

Tunable knobs (v0.4 onward)

`n_planes`, `n_helos`, `n_crews`, `n_dozers`, `n_engines`, `doctrine` $\in$ {`auto`, `direct`, `indirect`}, `protect_assets`.

Physical constraints are enforced

Fleets degrade above `DEGRADE_WIND_MS` $= 13$ m/s and are **grounded** at `GROUND_WIND_MS` $= 18$ m/s — so a 25 m/s high-wind Eaton run is genuinely unchanged by air support.

Source: `CHANGELOG.md`, `agents/firefighter_core.py`.

EXISTING RESULTS

Calibration Against Real Fires

The credibility argument

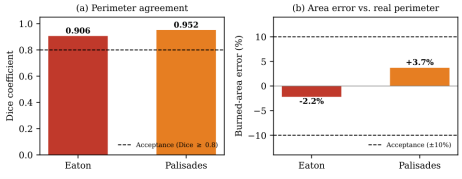


Figure 1 from "When Wildfire Breaks the Grid".

Fire	Dice	Area err.	Verdict
Eaton	0.906	$-2.2\,\%$	PASS
Palisades	0.952	$+3.7\,\%$	PASS

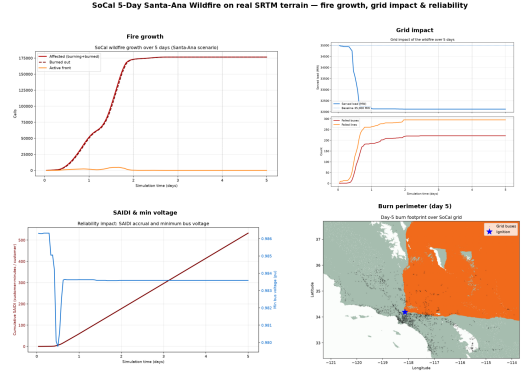
- Acceptance bar: Dice ≥ 0.8 **and** $|\text{area}\,\%| \leq 10$. Both fires clear it.
- Reference: official CAL FIRE perimeters at env step 60. Eaton 13,786 vs 14,102 ac; Palisades 24,677 vs 23,799 ac.

Honest caveat

Without containment the free-running CA *overshoots* by $+145\,\%$ (Eaton) and $+129\,\%$ (Palisades). The match relies on `wind_field.py`: perimeter-informed wind, fuel reclassification and `contain_burnable_footprint(..., margin_cells=2)`.

Five-Day Santa-Ana Baseline

The headline scenario



analysis/five_day_combined.png

Setup — 120 hourly steps, seed 47, synthetic 600×760 raster. Strong dry offshore wind on days 1–3, marine-layer recovery on days 4–5.

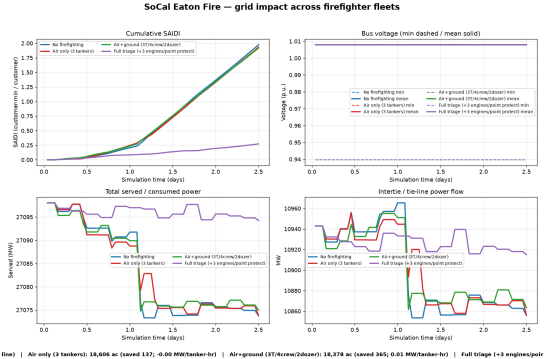
Outcome

- Served load ≈35,000 MW → **31,328 MW**
- ≈213 buses and ≈327 lines de-energised
- Peak ≈**734,000 customers** disconnected, around day 2.5
- Final burn footprint ≈166,000 cells
- Power flow converged **120/120** steps

Reproduce with `python analysis/run_5day.py --max-steps 120`.

Suppression Counterfactuals

Four phases of incident response



analysis/grid_metrics_v04.png

Phase	Acres saved	SAIDI
phase_0_no_ff	—	≈1.97
phase_1_air	137	≈1.95
phase_2_air_ground	365	≈1.93
phase_3_full_triage	**654**	**0.27**

The key insight of the whole project

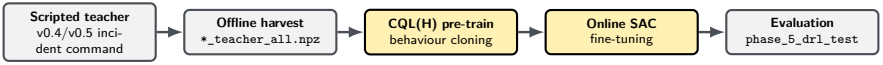
Full triage cuts total burned area by only a few percent — yet it drops SAIDI from ≈1.97 to **0.27** and keeps ≈20 MW more load energised.

A tactic that barely moves total burned area can still be decisive for grid resilience — if it protects the right cells.

On the Eaton fine grid, full triage holds SAIDI 1.61 → 0.24 (≈5,588 ac saved); Palisades ≈1,811 ac saved.

The Learning Firefighter

v0.7 — current frontier



The learning contract

Box(17) observation — burning/burned/suppressed fractions, fire-front geometry, mean slope, wind trigonometry, served MW, SAIDI delta, step progress. *Not* a raster flatten.

Discrete(4) action — `NOOP`, `INDIRECT`, `DIRECT`, `TRIAGE`. The policy picks a *doctrine*; the same IncidentCommand machinery executes it, gated by resource availability.

Reward — `SaidiObjective`: $r = -\Delta\text{SAIDI}/\text{scale}$, always ≤ 0, exactly 0 when load is fully served.

Why gate the actions?

So the learner *cannot invent impossible suppression*. It chooses tactics, not physics.

State this honestly

The v0.7 verification was a **short pipeline smoke run on synthetic-terrain fallback** — not a real-DEM result. Those SAIDI numbers are *not* metric-comparable to production, and two episodes cannot train a policy.

Production long-runs are configured (`episodes: 400`, `cql_alpha: 1.0`, `use_real_dem: true`) but not yet reported.

Two Bugs Worth Knowing About

Because you will hit their cousins

1 — Ragged sensor arrays

Symptom. palaestrAI's `Memory` crashed with `ValueError: All arrays must be of the same length`.

Cause. `gis.cell_state` (≈23,660 elements) was mixed with scalar `*-load-*.p_mw` sensors in one DataFrame.

Fix. `agents/_memory_compat.py`, a ragged-safe, idempotent shim for `_MuscleMemory._infos_to_df` — installed in *both* the RolloutWorker *and* the Learner process.

2 — The silent no-op learner

Symptom. hARL logged "could not update"; the firefighter trained for hours *without ever learning*.

Cause. Action-shape mismatch. The muscle emitted `np.array([act_id])` with shape `(1,)`, while the offline loader stored a 0-d scalar. `SACBrain.update()` builds a single `np.array(actions)`, which then went ragged.

Lesson. A DRL agent that silently fails to update looks exactly like a DRL agent that has not converged yet. Assert on your shapes.

Source: `CHANGELOG.md`, v0.7.

RUNNING THE TESTBED

Quick Start

Fifteen minutes to your first figure

1 — Install and verify

```
pip install -r requirements.txt      # or requirements-dev.txt
pytest -m unit                       # < 5 s, numpy only
pytest -m slow                       # loads the 5.9 MB grid, runs power flow
```

2 — Reproduce the headline result (no broker, no database)

```
python analysis/run_5day.py --max-steps 120 --outdir analysis
python analysis/make_timelapse.py     # -> wildfire_timelapse.gif + .mp4
```

3 — Optional: real terrain (free OpenTopography key)

```
export OPENTOPOGRAPHY_API_KEY=...
python data/dem/fetch_dem_tiles.py    # -> socal_srtm_gl3.npz, ~71 MB, git-ignored
```

`run_5day.py` is a standalone driver. It writes `FIVE_DAY_ANALYSIS.md`, four PNGs and `five_day_kpis.csv` — and needs no palaestrAI runtime at all.

The Full palaestrAI Path

When you need agents, phases and a store

Standard run

```
python midas_socal/prepare_midas.py

palaestrai database-create

# validate the YAML before burning an hour
python -c "from palaestrai.experiment \
  import ExperimentRun; \
  ExperimentRun.load(
    'palaestrai_socal/experiment.yml')"

palaestrai start palaestrai_socal/experiment.yml
```

MIDAS scenario only

```
midas_socal/run_sim.sh            # full
midas_socal/run_sim.sh --smoke    # CI / quick
```

Multi-phase A/B comparison

```
env PYTHONPATH=$PWD palaestrai \
  -c runtime_pg_eaton.conf.yaml start \
  palaestrai_socal/experiment_eaton_local_ab.yml
```

`--phases` grammar

Comma-separated list; each item is `phase_uid`, `phase_uid:n_planes`, or `uid:n_planes:Label`.

No commas inside a label. Legacy `--phase-a`/`--phase-b` still work.

Pick a `runtime*.conf.yaml` to match your horizon: `runtime_short`, `runtime_full16`, `runtime_full120`.

Key Scripts and What They Produce

Your analysis toolbox

Script	Purpose	Output
`analysis/run_5day.py`	Standalone 120-step baseline run	report + 4 PNGs + KPI CSV
`analysis/verify_calibration.py`	Assert Dice and area error against CAL FIRE perimeters	pass/fail verdict
`analysis/perimeter_validation.py`	Compare simulated vs. official perimeter geometry	Dice, area, overlay
`analysis/grid_metrics_report.py`	SAIDI, voltage, served power across phases	`grid_metrics_*.png`
`analysis/make_timelapse.py`	Animate a single run over the Esri satellite basemap	GIF + MP4
`analysis/make_comparison_timelapse.py`	Side-by-side animation of several phases	GIF + MP4
`analysis/make_sim_vs_real.py`	Simulated vs. real perimeter figure	comparison PNG
`analysis/firefighter_report.py`	Scripted-suppression KPI summary	tables + figures
`analysis/drl_firefighter_report.py`	Learned vs. scripted responder	comparison report

Scripts that read a store take `--store <uri>` and `--phases`; standalone drivers do not. Basemap tiles are cached in `data/basemap_cache/` — no API key needed. Attribution: Esri, Maxar, Earthstar Geographics.

Reproducibility and Provenance

Read before you upgrade anything

Pinned for a reason

palaestrAI constrains `numpy<2` and `pandas==2.1.4`. Do **not** bump either without re-running the calibration verification — the CA is validated against a specific numerical stack.

Determinism

The only stochastic element is the seeded neighbour-ignition draw. Same seed and same Θ give the same fire, every time.

CI pipeline (`.gitlab-ci.yml`)

`lint`	flake8, on push
`unit`	`pytest -m unit`, on push
`system`	`pytest -m slow`, manual
`simulate`	NOAA smoke + 5-day, manual

`simulate` publishes its figures as artifacts.

Open data only

USGS 3DEP / SRTM GL3, LANDFIRE FBFM40, OSM, CAISO, NOAA HRRR, CAL FIRE perimeters. GPL-3.0-or-later.

ARL RESEARCH QUESTIONS

From Calibrated Substrate to Adversarial Learning

Where the open work begins

Today

Θ is a **calibrated constant**, set once from the validation values and held fixed. The wildfire agent uses `DummyBrain`: it acts, it never learns.

The published paper says this explicitly — the testbed is *"the calibrated, validated substrate on which a future learned-scenario-generation loop could be built."*

The ARL target

Replace the constant with a **learned Overseer-Adversary** that proposes Θ to maximise regret against the operating agent, subject to an *admissibility predicate* $\mathcal{A}_{KG}$ that keeps every proposed world physically plausible.

Both sides then improve together — an autocurriculum in the sense of Baker et al. (2020) and unsupervised environment design.

Why this repository is the right starting point

The hard part — a fire model whose output is *trustworthy* (Dice ≥ 0.9 against two real perimeters) and coupled to a solvable grid — is already done and test-guarded. What is missing is the learning loop around Θ.

Research Questions

Available as thesis topics

RQ1 — Adversarial scenario generation

How can Θ be *learned* rather than calibrated? Which admissibility constraints keep generated worlds physically plausible while still being genuinely hard?

RQ2 — Objective design for resilience

Which signal best represents grid resilience: SAIDI, served MW, fraction of customers connected, or time-to-recovery? How do learned policies differ under each, in the resilience-trapezoid metric space?

RQ3 — Out-of-distribution generalisation

Can a responder trained on Eaton transfer to Palisades, to synthetic fires, or to a different hazard entirely — and how do we measure that honestly?

RQ4 — Reward hacking and simulator artifacts

The suppression action space is deliberately gated. Where else could a policy exploit the model rather than the phenomenon, and how would we detect it?

Research Questions

Continued

RQ5 — The mutation-operator boundary

Where should the line sit between a *constrained mutation operator* (cheap, interpretable, provably admissible) and a *learned adversary* (expressive, but capable of drifting into the implausible)?

RQ6 — Catastrophic forgetting in multi-phase crises

A real crisis is sequential: fire, then outage, then restoration. How do we keep an agent competent at phase 1 while it learns phase 3? (Kirkpatrick et al., 2017.)

RQ7 — Cross-hazard reuse

`gis_world` is hazard-agnostic by construction. What does it take to add δ_{wind}, δ_{flood} or δ_{grid} from the GUARDIAN operator bank — and do policies trained on fire transfer at all?

A good first paper

Take RQ2. It needs no new physics — only a new `Objective` subclass and a careful comparison. The store already holds everything you need.

Summary and Your First Contribution

What you now have

- A fire model **validated** against two real 2025 perimeters (Dice 0.906 / 0.952).
- Bidirectional **fire-to-grid coupling** with a full power-flow re-solve each step.
- Four scripted suppression tiers as **counterfactuals**, plus a first learning responder.
- **235 passing tests**, open data only, GPL-3.0-or-later.
- A clear, documented **ARL extension path**.

Pick one, this week

1. **Reproduce.** Run `run_5day.py` and match the published figures.
2. **Read.** Open `wildfire_cma/cma.py` and find equations 6 and 7 in the code.
3. **Perturb.** Change one component of Θ and explain the effect on SAIDI.
4. **Propose.** Write half a page on one of RQ1–RQ7.

Repository

`gitlab.com/arl-experiments/socal-wildfires`

References and Further Reading

- V. Sultan, T. Logemann, E. M. S. P. Veith. *GUARDIAN: Geospatial Unseen-event Adversarial Reinforcement for Defense and Infrastructure Adaptation.*
- E. M. S. P. Veith, V. Sultan. *When Wildfire Breaks the Grid: A Validated Extreme-Event Testbed for Infrastructure Resilience.*
- R. C. Rothermel (1972). *A mathematical model for predicting fire spread in wildland fuels.* USDA Forest Service.
- C. Pais et al. (2021). *Cell2Fire: A cell-based forest fire growth model.*
- M. Panteli et al. (2017). *Power system resilience assessment* — the resilience trapezoid.
- M. Bruneau et al. (2003). *A framework to quantitatively assess and enhance seismic resilience of communities.*
- J. Kirkpatrick et al. (2017). *Overcoming catastrophic forgetting in neural networks.*
- B. Baker et al. (2020). *Emergent tool use from multi-agent autocurricula.*
- E. M. S. P. Veith et al. (2023). *palaestrAI* — the experiment platform underlying this testbed.
- IEEE Std 1366 (2012). *Guide for electric power distribution reliability indices* — the SAIDI definition used throughout.

QUESTIONS?

gitlab.com/arl-experiments/socal-wildfires

eric.veith@offis.de | vsultan3@calstatela.edu